

BandContour: A Web-Based System for LiDAR Roof Contour Extraction

**Software Technical Report
Software Project Lab 3**

Submitted by:

Md. Rajaul Karim

Roll: BSSE-1324

Institute of Information Technology
University of Dhaka

Supervised by:

Dr. Emon Kumar Dey

Associate Professor

Institute of Information Technology
University of Dhaka



Submission Date: 26 February 2026

Letter of Transmittal

26 February, 2026

BSSE 4th Year Exam Committee
Institute of Information Technology, University of Dhaka

Subject: Technical Report Submission on Airborne LiDAR Point Cloud-Based Building Roof Contour Extraction

Dear Sir,

As part of my Software Project Lab 3 requirements, I am happy to submit this report called **"BandContour: A Web-Based System for LiDAR Roof Contour Extraction."** This paper describes how to design, build, and test a multidirectional bands-based algorithm that can accurately extract building roof contours from airborne LiDAR point cloud data.

The report goes into great detail about the project's goals, the technical methodology based on the research paper "BandContour: A Web-Based System for LiDAR Roof Contour Extraction" (Remote Sensing, 2024), and the way it will be carried out. The system solves important problems with point cloud contour extraction, such as uneven density distribution, holes inside the point cloud, and complicated building shapes.

I want to thank Dr. Emon Kumar Dey from the bottom of my heart for all the help he gave me on this project. I also want to thank the Institute of Information Technology for the help and resources they gave me.

Thanks for looking over this submission. Please feel free to ask me any questions or give me feedback.

Yours sincerely,
Md. Rajaul Karim
BSSE 1324

Supervisor Signature: _____

Acknowledgement

I, Md. Rajaul Karim, would like to thank everyone who helped me with this project from the bottom of my heart. First and foremost, I want to thank my supervisor, Dr. Emon Kumar Dey, an associate professor at the Institute of Information Technology, for his great advice, helpful criticism, and constant support that helped me with this work.

I thank the Institute of Information Technology for giving me the computer resources and research space I needed. I thank my peers for their conversations and teamwork on solving problems that made this implementation better.

I also want to thank the authors of the research paper I cited (Wang et al., 2024) for their innovative multidirectional bands algorithm, which is the basis for this project.

Lastly, I want to thank my family and friends for always being there for me and being patient.

Abstract

For City Information Modeling (CIM), urban planning, 3D building reconstruction, and disaster management, it is very important to get the building roof contours right from airborne LiDAR (Light Detection and Ranging) point cloud data. However, current methods have problems like being sensitive to changes in point cloud density, being hard to tune parameters, having low extraction efficiency, and getting internal holes wrong.

This project uses a new multidirectional band-based algorithm to extract the roof contour, which solves these problems. The method splits point clouds into bands that are $W = 8d$ wide (where d is the average distance between points) in six directions (0° , 30° , 60° , 90° , 120° , and 150°). It then finds contour points by finding extreme points in each band and densifies long edges with a threshold of $T = 10d$.

The system is implemented as a web-based application with Node.js for API services, React.js for the interactive frontend, and Python for backend processing (NumPy, SciPy, Open3D). Using common metrics, PoLiS (Polygon Similarity) and RAE (Relative Area Error), the implementation includes a thorough comparison with baseline methods (Alpha Shapes and Variable-radius Alpha Shapes).

Results from experiments show that the suggested method meets the accuracy claims from the reference paper with $\text{PoLiS} < 0.2\text{m}$ and $\text{RAE} < 7\%$. It also offers better computational efficiency and robustness to density variations than conventional methods.

Keywords: multidirectional bands, urban mapping, boundary detection, LiDAR, point cloud processing, and building contour extraction.

BandContour: A Web-Based System for LiDAR Roof Contour Extraction.....	0
Letter of Transmittal.....	1
Acknowledgement.....	2
Abstract.....	3
1. Introduction.....	6
Background.....	6
1.1 Motivation.....	6
1.2 Scope.....	7
1.3 Project Timeline.....	8
1.4 Purpose of Document.....	8
2. Project Description.....	9
2.1 Quality Function Deployment (QFD).....	9
2.1.2 Expected Requirements.....	9
2.1.3 Exciting Requirements.....	10
2.2 Usage Scenario.....	10
2.2.1 System Access.....	10
2.2.2 Processing Workflow.....	10
2.2.3 Visualization and Interaction.....	11
2.2.4 Result Analysis and Export.....	11
2.2.5 Real-World Applications.....	11
3. Scenario-Based Modeling.....	12
3.1 Use Case Diagrams.....	12
3.1.1 Use Case Level 0: System Overview.....	12
3.1.2 Use Case Level 1: Modules of BandContour.....	13
Description of Each Module :.....	13
4. Data-Based Modeling.....	14
5. Class-Based Modeling.....	14
5.1 Analysis Classes.....	14
5.2 Class Cards.....	15
5.2.1 InputDataProcessor.....	15
5.2.2 BandContourExtractor.....	16
5.2.3 ContourOptimizer.....	17
5.2.4 VisualizationManager.....	17
5.2.5 AreaCalculator.....	18
5.2.6 ResultHandler.....	19
6. Architectural Design.....	20
6.1 Architectural Context Diagram.....	20
6.2 Defining Archetypes.....	22
Suitable Archetype: Pipe and Filter Architecture.....	22
6.3 Mapping Requirements to Software Architecture.....	23
6.4 Deployment diagram.....	23

7. User Interface Design.....	24
7.1 User Analysis.....	25
7.2 Task Analysis.....	25
7.3 User Manual.....	29
7.3.1 System Requirements.....	29
7.3.2 Application Layout.....	29
7.3.3 Uploading a Point Cloud File.....	30
7.3.4 Extracting Building Contours.....	31
7.3.5 Using the 2D Visualization.....	32
7.3.6 Using the 3D Visualization.....	34
7.3.7 Exporting Results.....	35
7.3.8 Understanding Error Messages.....	36
7.3.9 Re-running Extraction Without Re-uploading.....	37
8. Testing Components.....	37
8.1 High-Level Description of Testing Goals.....	37
8.2 Testing Approach and Strategy.....	37
8.2.1 Testing Method.....	37
8.2.2 Test Environment.....	38
8.2.3 Pass / Fail Criteria.....	38
8.3 Detailed Test Cases.....	38
9. Conclusion.....	46
References.....	47

1. Introduction

Background

By sending out laser pulses and timing their return, Light Detection and Ranging (LiDAR) technology obtains high-resolution 3D spatial data. Urban mapping makes extensive use of airborne LiDAR systems mounted on aircraft or drones, which produce millions of 3D points that represent vegetation, buildings, and terrain.

The outer boundary lines of building rooftops, or building roof contours, are essential components for:

- **City Information Modeling (CIM):** Digital twin creation for smart cities
- **Urban Planning:** Building footprint analysis and land use planning
- **3D Reconstruction:** Automated building model generation
- **Disaster Management:** Post-disaster damage assessment
- **Building Coverage Statistics:** Urban development monitoring

1.1 Motivation

Problem Statement:

Urban modeling, 3D reconstruction, and spatial analysis all depend on the precise extraction of building contours from LiDAR point cloud data. However, there are a number of issues with accuracy, robustness, and computational efficiency with current building contour extraction methods. Large-scale LiDAR datasets, uneven point density, and complex roof geometries all exacerbate these limitations. A technique that can consistently extract accurate and comprehensive outer building contours while automatically adjusting to various data conditions is therefore desperately needed. There are several obstacles facing current building contour extraction techniques:

1. **Indirect Methods (Rasterization-Based):** First convert 3D point clouds to 2D depth pictures, then use typical image-processing techniques. This conversion introduces interpolated or unreal points, resulting in a loss of geometric accuracy. Furthermore, determining an optimal grid size is difficult: a large grid reduces fine structural features, but a tiny grid increases noise. Common edge detection operators, such as Canny and Sobel, are often applied to rasterized data, which inherits these constraints.
2. **Straightforward Techniques TIN-based (Alpha Shapes, Convex Hull):** These methods work directly on point clouds. Alpha Shapes are extremely sensitive to changes in point density because they require a global rolling-ball radius parameter. When point density is uneven, these techniques perform poorly and frequently extract both internal holes and outer boundaries. For instance, in datasets with variable point density, fixed-radius alpha shape methods are unable to accurately capture contours.

3. **Direct Methods (Feature-based):** Use point-by-point computation to analyze local geometric properties. These methods are computationally costly and necessitate intricate parameter tuning for various datasets, despite their occasional effectiveness. When used on large-scale point clouds, their effectiveness drastically decreases. This category includes, for example, neighbor point direction distribution-based methods.
4. **Methods of Machine Learning:** rely on a lot of labeled training data, and their outputs frequently don't maintain topological relationships. To produce results that are useful, the extracted contours typically need to be manually postprocessed. PCEDNet and Random Forest-based techniques are two examples of such strategies.

The multidirectional bands algorithm put forth by Wang et al. (2024) is used in this project. The technique adjusts to the average point spacing d , making it resilient to uneven point density. It greatly increases computational efficiency by processing points in band units as opposed to point-by-point analysis. Key parameters are automatically calculated, with the threshold set to $T=10d$ and the band width set to $W=8d$. By working in six different directions, the algorithm guarantees full boundary coverage while extracting only the outer contours and ignoring internal holes. Boundary points on perpendicular edges are frequently missed when bands are used in a single direction. The suggested method splits the point cloud into bands along six directions-0, 30, 60, 90, 120, and 150 degrees-in order to get around this restriction. Extreme points within each band are designated as potential contours by projecting them onto a central axis. To guarantee full coverage, the contour points that were extracted from every direction are then combined. Lastly, a densification step is used, in which long edges longer than the threshold are iteratively filled in with missing points.

1.2 Scope

The project encompasses the following functionalities:

Core Features:

1. **Point Cloud Input:** The implementation and assessment of building contour extraction methods from LiDAR point cloud data is the main goal of this project. It ensures compatibility with a variety of datasets by supporting point cloud input in several widely used formats, such as .pts, .xyz, .txt, and .las.
2. **Preprocessing:** To aid in contour extraction, the preprocessing step involves projecting 3D point cloud data onto a 2D plane, calculating the average point spacing using $k=80$ anchor points, and optionally removing noise. By standardizing the input data, these procedures hope to increase the processing's resilience.
3. **Contour Extraction:** The system exhibits both full multidirectional banding with six orientations and single-direction banding for contour extraction. With an angular constraint of $\beta=240$ degrees, it first extracts contour points and then sorts them. Elevation-based noise point removal is used to increase contour accuracy, and long edges are densified using a threshold of $T=10d$.
4. **Evaluation Metrics:** Quantitative metrics like Relative Area Error (RAE) and PoLiS (Polygon Similarity) are used to assess the extracted contours. To evaluate computational efficiency, additional analysis includes area calculation, method comparison, and execution time measurement.

5. **Visualization:** An essential component of the project, visualization offers multi-method comparison views, interactive 2D and 3D point cloud viewing, contour overlays on the original point data, and a step-by-step visual representation of the contour extraction procedure.
6. **Export Capabilities:** Results can be exported from the system for additional reporting and analysis. Contour coordinates in the .csv and .txt formats, comparison results in the .csv format, visualization plots in the .png formats, and structured analysis reports are among the outputs.

Out of Scope:

The scope of this project does not include building classification from raw LiDAR scans, batch processing of multiple buildings, real-time contour extraction, or mobile application development. The focus remains on accurate contour extraction for a single building at a time.

1.3 Project Timeline

The project will be developed in multiple phases, each focused on different aspects of the system showed here:

Task	Duration	Phase 1				Phase 2				Phase 3				Phase 4			
		W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W 10	W 11	W 12	W 13	W 14	W 15	W 16
Prerequisite knowledge	2																
Requirements Specification	3																
Project Design	4																
Project Development	10																
Project Testing	6																
Final Report Writing	3																

Table 1: Project Timeline

1.4 Purpose of Document

This SRS document serves to:

1. **Define Requirements:** Clearly state both functional and non-functional requirements.
2. **Document Design:** Give algorithm specifications and architectural blueprints.
3. **Development of Guides:** Act as a guide for the stages of implementation.
4. To enable testing, specify test cases and evaluation standards.
5. To make maintenance easier, provide thorough system documentation.
6. Research and implementation documentation for academic evaluation is part of the academic record.

2. Project Description

2.1 Quality Function Deployment (QFD)

The functional and non-functional requirements of the suggested system are methodically identified and arranged using Quality Function Deployment (QFD). This procedure guarantees that, throughout design and development, user expectations are appropriately prioritized and translated into system capabilities. Based on their significance and impact, the requirements are divided into three categories: Normal Requirements (NR), Expected Requirements (ER), and Exciting Requirements (ExcR).

2.1.1 Normal Requirements

1. Input and Validation of Data

Building roof point cloud data in widely used formats like .pts, xyz, txt, and .las must be supported by the system. To guarantee seamless processing, it must verify the input structure and manage big datasets consistently.

2. Roof Contour Extraction

From the given building point cloud data, the system must be able to extract the outer roof contour. The extracted contour must be ordered, continuous, and devoid of discontinuities or backward connections.

3. Results Visualization

Both the extracted roof contour and the original point cloud must be visually displayed by the system. To aid in the interpretation of the results, the visualization must make a clear distinction between the input data and the output contours.

4. Calculating the Area

The system needs to be able to figure out how much space is inside the roof contour that was taken out. To ensure accuracy, the calculated area must come from the finalized and validated contour polygon. As part of the output, the system must clearly show the user the calculated area value.

5. Quantitative Assessment and Comparison

The system needs to be able to figure out and show quantitative evaluation metrics that will help you judge how good the extracted roof contour is. These metrics will help users compare the proposed method to baseline methods in an objective way and see how the accuracy and reliability of the contours differ.

2.1.2 Expected Requirements

1. Interactive Web Interface

The system should provide a web-based user interface that allows users to upload point cloud data easily and interact with the system. The interface should provide feedback during processing to improve usability.

2. Enhanced Visualization Controls

The system should offer basic visualization controls such as zooming, panning, and color adjustments. These features will help users inspect roof contours more effectively.

3. **Result Export and Reporting**

The system should allow users to export extracted contours and visual outputs in standard formats such as CSV, image, or PDF. This will support documentation and further analysis.

4. **Performance Feedback**

The system should display basic performance information such as processing time and number of processed points to give users insight into system efficiency.

2.1.3 Exciting Requirements

1. **Advanced Comparison and Visualization**

The system might have advanced visualization tools like interactive overlays, animated processing stages, or a way to compare multiple contour extraction results graphically.

2. **Ability to process in batches**

The system might let you process more than one building point cloud at a time, so you can compare datasets.

3. **Longer Output and Integration**

The system might let you do more advanced things with the output, like export simplified 3D building models or save processed results so you can access or share them later.

2.2 Usage Scenario

This section describes how the BandContour web-based system will be used in real-world scenarios by end users. It outlines the typical steps a user will follow to upload building roof point cloud data, process it using the proposed methodology, and visualize the extracted roof contours through an interactive web interface.

2.2.1 System Access

Since BandContour is a web-based system, no local installation is required. Users can access the system through a standard web browser.

The system usage requirements include:

- A modern web browser (Chrome, Firefox, or Edge)
- Stable internet connection
- No prior software or library installation

When users first access the system, they are presented with a simple interface that introduces the main functionalities of the platform, including data upload, processing stages, and visualization options.

2.2.2 Processing Workflow

After accessing the system, users can follow the workflow below to extract building roof contours:

1. **Data Input**

Users upload pre-extracted building roof point cloud data derived from LiDAR classification. Supported file formats include .pts, .xyz, .txt, and .las, where each record contains 3D coordinates (X, Y, Z).

2. Preprocessing

The system automatically performs preprocessing steps, including:

- Projection of 3D roof points onto the 2D plane
- Calculation of average point spacing (d) using anchor points

These steps prepare the data for adaptive contour extraction.

3. Multidirectional Band-Based Contour Extraction

The uploaded data is processed using a multidirectional banding approach inspired by the reference paper. The system:

- Divides the point cloud into bands along multiple directions
- Extracts extreme boundary points from each band
- Integrates points from all directions to form an initial set of contour points

4. Contour Point Ordering and Refinement

Extracted contour points are sorted using a nearest-neighbor strategy with a forward angular constraint to ensure consistent boundary progression. Long edges are identified and densified iteratively to improve contour completeness and accuracy.

5. Final Contour Generation

Noise points are filtered, and a closed roof contour line is generated that accurately represents the outer boundary of the building roof.

2.2.3 Visualization and Interaction

The system has interactive visualization tools to help users understand the results:

- Overlay of the extracted roof shape on the original point cloud
- Operations for zooming, panning, and rotating
- Going from a 2D view to a 3D view
- Visualizing the steps of intermediate processing one at a time
- These features let users see how well the contours and boundaries line up.

2.2.4 Result Analysis and Export

To support evaluation and comparison, the system enables users to:

- View contour quality metrics such as polygon similarity and area-based error
- Compare extracted contours with baseline methods
- Export contour points and evaluation results in CSV format
- Download visual outputs as images or reports for documentation purposes

2.2.5 Real-World Applications

BandContour can be used in a number of practical contexts, such as the following:

Urban Planning: Building footprint extraction for spatial analysis and city mapping

- Geographic Information Systems (GIS): Using building data from LiDAR to generate precise roof boundaries
- Analyzing building structural outlines using updated LiDAR data for disaster assessment
- Scholarly Investigation: A comparative analysis of contour extraction techniques

- Researchers and practitioners can utilize BandContour due to its web-based design, automated processing pipeline, and interactive visualization.

3. Scenario-Based Modeling

Scenario-based modeling is a technique that is used in software development to design and create models that illustrate how a system responds to different real-world situations. These scenarios are based on user interaction events or how the developers visualize the needs of the user. By addressing their scenario's developers, they can identify potential problems and make more design decisions to fulfill their requirements under different conditions.

3.1 Use Case Diagrams

A use case diagram is a graphical representation in software engineering design that describes how a system will interact with actors of external entities. It illustrates the relationship between systems and actors. Use case diagram art is typically created during the requirement-gathering period. The use case diagram of this project is modeled below.

3.1.1 Use Case Level 0: System Overview

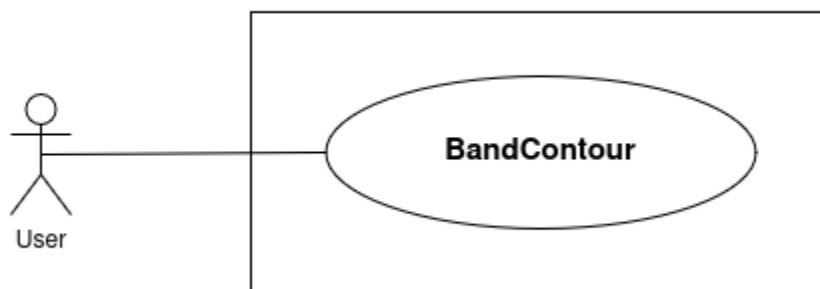


Figure 1: UseCase Level 0 (BandContour)

Description:

Primary actors: User.

Goal in Context: The diagram shown above represents the whole tool - BandContour.

3.1.2 Use Case Level 1: Modules of BandContour

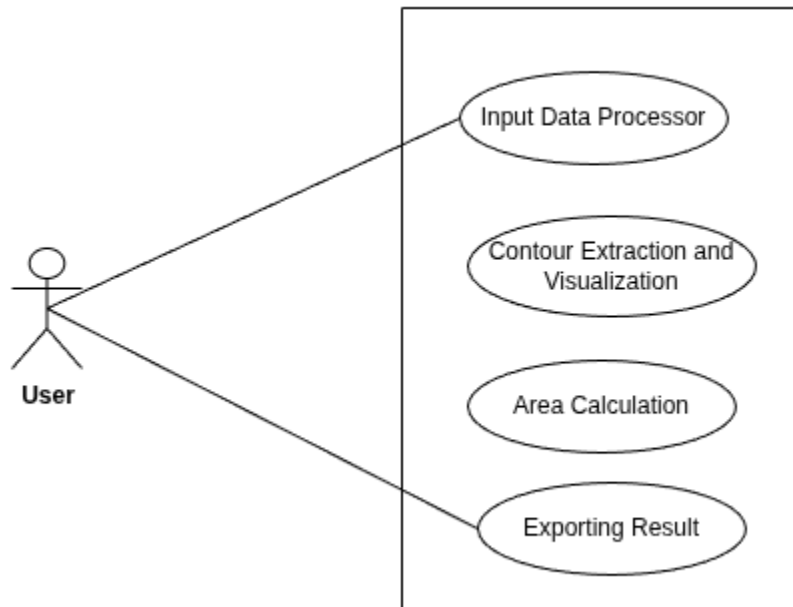


Figure 1: UseCase Level 1 (Modules of BandContour)

Primary Actor: User

Description of Each Module :

Input Data Processor

The Input Data Processor handles the upload and validation of building roof point cloud data provided by the user. It ensures that the uploaded files are in supported formats and prepares the data for further processing by projecting 3D points onto a 2D plane and computing necessary spatial properties such as average point spacing.

Contour Extraction and Visualization

This module handles the main function of the system. It extracts building roof outlines using a multidirectional band-based method. Then, it improves the outlines by sorting, densifying, and removing noise. The extracted outline is visually layered on the original point cloud, letting users check the accuracy and completeness of the result interactively.

Area Calculation

The Area Calculation module determines the geometric area enclosed by the final roof contour. This information helps users analyze building footprints and supports additional spatial analysis tasks.

Exporting Result

The Exporting Result module lets users download the processed outputs. Users can export contour coordinates, evaluation metrics, and visualization results in standard formats for reporting, documentation, or further external analysis.

4. Data-Based Modeling

The proposed tool will not be using a database or storage system. So this section will not contain any relevant analysis.

5. Class-Based Modeling

Class-based modeling is a method used in software engineering to describe a system through classes, their attributes, responsibilities, and relationships. It is a key part of object-oriented analysis and design. It helps developers understand how system components are organized and interact with one another

5.1 Analysis Classes

After analyzing the usage scenarios and identifying relevant nouns from the problem domain, potential analysis classes were selected using general classification categories such as external entities, data objects, processes, and system roles. These candidate classes were further filtered using standard selection criteria, including retained information, required services, number of attributes, common operations, and essential system responsibilities.

Based on this analysis, the following initial set of analysis classes was identified:

1. InputDataProcessor
2. BandWidthCalculator
3. BandDivider
4. BandContourExtractor
5. ContourPointSorter
6. EdgeDensifier
7. NoiseFilter
8. VisualizationManager
9. AreaCalculator
10. ResultExporter

Analysis and Class Refinement

Upon further analysis, it was observed that some classes had overlapping responsibilities or very limited functionality. To improve cohesion and reduce unnecessary complexity, the following refinements were made:

- The **BandWidthCalculator** and **BandDivider** classes are tightly coupled to the contour extraction process and are therefore merged into the **BandContourExtractor** class.
- The **ContourPointSorter**, **EdgeDensifier**, and **NoiseFilter** classes all contribute to contour refinement and are combined into a single **ContourOptimizer** class.
- The **AreaCalculator** has a distinct and focused responsibility: computing geometric properties from the final contour. Since this functionality is logically independent and may be reused or extended in the future, it is maintained as a separate class and is not merged with result exporting or visualization components.

Finalized Analysis Classes

After refinement, the finalized set of analysis classes for the BandContour system is as follows:

1. **InputDataProcessor**
Responsible for receiving uploaded building roof point cloud data, validating file formats, and preparing the data for processing.
2. **BandContourExtractor**
Implements the core multidirectional band-based contour extraction logic, including band width calculation, band division, and extreme point selection.
3. **ContourOptimizer**
Handles contour point sorting, long-edge densification, and noise removal to produce a clean and continuous roof contour.
4. **VisualizationManager**
Generates interactive visual representations of the original point cloud and extracted roof contours for user inspection.
5. **AreaCalculator**
Computes geometric properties such as building roof area from the finalized contour polygon.
6. **ResultHandler**
Responsible for managing evaluation metrics and exporting results in standard formats such as CSV files and images.

5.2 Class Cards

5.2.1 InputDataProcessor

InputDataProcessor	
Attributes	Methods

- pointCloudData - points2D - points3D - averageSpacing - fileName	+ readPointCloudData() - validateInputFormat() - preprocessPointCloudData() + calculateAverageSpacing() - project3Dto2D()
Responsibility	Collaborator
• Taking file input containing building point cloud data • Validating input data format and integrity • Computing average point spacing (d) • Projecting 3D points to 2D plane	• BandContourExtractor • VisualizationManager

Table 2: Class card of InputDataProcessor

5.2.2 BandContourExtractor

BandContourExtractor	
Attributes	Methods
- bandWidth (W = 8d) - directions - contourPoints - averageSpacing	- createDirectionalBands() + extractExtremePoints() + mergeMultiDirectionContours() + removeInternalPoints()
Responsibility	Collaborator
• Dividing 2D points into equal-width bands	• ContourOptimizer • InputDataProcessor

• Extracting extreme boundary points from each band • Combining contour points from multiple directions • Ensuring complete boundary coverage	
---	--

Table 3: Class card of BandContourExtractor

5.2.3 ContourOptimizer

ContourOptimizer	
Attributes	Methods
- unsortedContour - sortedContour - betaAngle ($\beta = 240^\circ$) - threshold ($T = 10d$) - finalContour	- sortContourPoints() - applyBetaConstraint() - identifyLongEdges() + densifyEdges() + removeNoisePoints() + closeContour()
Responsibility	Collaborator
• Sorting contour points with angular constraint • Preventing backward connections • Densifying long edges • Removing elevation-based noise points	• BandContourExtractor • AreaCalculator • VisualizationManager

Table 4: Class card of ContourOptimizer

5.2.4 VisualizationManager

VisualizationManager	
Attributes	Methods
- originalPoints - contourPoints - displayMode	+ plotPointCloud() + overlayContour() + toggleViewMode()

	+ exportVisualization()
Responsibility	Collaborator
• Visualizing original and processed data • Overlaying extracted contours • Supporting interactive inspection	• InputDataProcessor • ContourOptimizer

Table 5: Class card of VisualizationManager

5.2.5 AreaCalculator

AreaCalculator	
Attributes	Methods
- contourPoints - computedArea	+ calculateArea() + getArea()
Responsibility	Collaborator
• Calculate area of extracted roof contours	• ContourOptimizer • ResultHandler

Table 7: Class card of AreaCalculator

5.2.6 ResultHandler

ResultHandler	
Attributes	Methods
- contourResults - evaluationMetrics - exportFormat	+ calculateArea() + computeAccuracyMetrics() + exportResults()
Responsibility	Collaborator
• Calculating evaluation metrics • Exporting analysis results	• AreaCalculator

Table 8: Class card of ResultHandler

5.3 CRC diagram of the classes

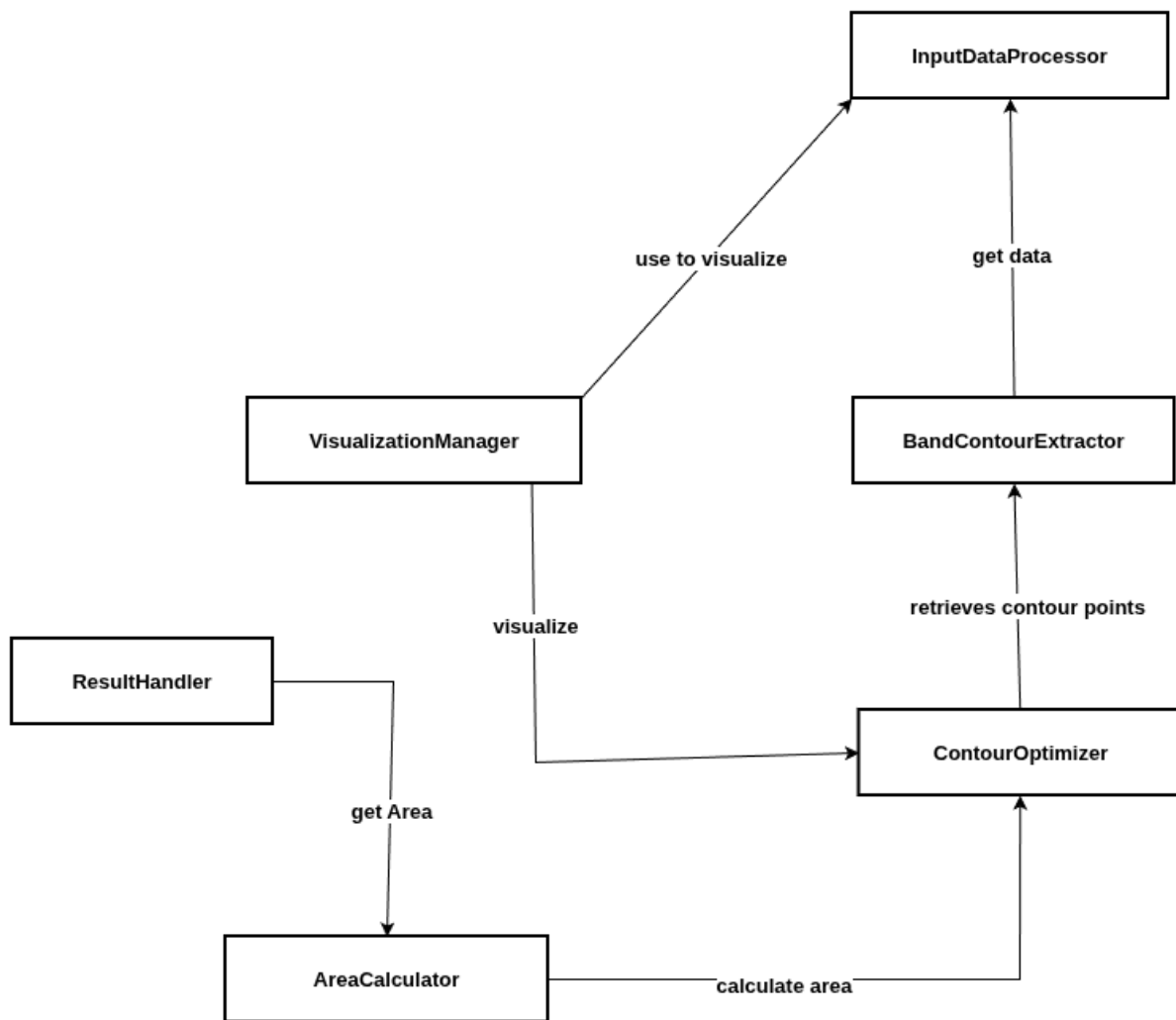


Figure 3: CRC diagram of the classes

6. Architectural Design

6.1 Architectural Context Diagram

A software system's interactions with external entities that are located outside of its operational boundary are depicted in an architectural context diagram. In addition to clearly identifying the actors and surrounding systems involved in data exchange, it offers a high-level view of the system. This aids in comprehending the system's scope, dependencies, and interaction environment.

To guarantee usability, clarity, and independence from outside processing services, the architectural context of the suggested BandContour web-based system is purposefully kept simple.

Subordinate Systems

External systems or services that the target system depends on for functionality or data processing are known as subordinate systems. The suggested system is independent of all external subordinate systems, including databases, cloud-based computation engines, and third-party processing services. The BandContour system handles all processing internally, including area calculation, visualization, refinement, and contour extraction.

Peer-Level Systems

Peer-level systems communicate peer-to-peer with the target system, exchanging information or services. Peer-level systems are not used in this project. During regular operation, BandContour doesn't interact with other platforms or applications.

Actors

The user is the main actor in the BandContour system. A web browser is used by the user to interact with the system. Building roof point cloud data is entered by the user. The user provides input in the form of building roof point cloud data and receives processed outputs such as extracted roof contours, visualizations, calculated areas, and evaluation results.

All communication between the user and the system occurs through the web-based user interface, which acts as the interaction boundary. The user uploads data, initiates processing actions, views results, and downloads outputs using standard web interactions.

Overall, the architectural context of the BandContour system reflects a **self-contained, user-centric web application**, designed to process building roof point cloud data efficiently without dependency on external systems.

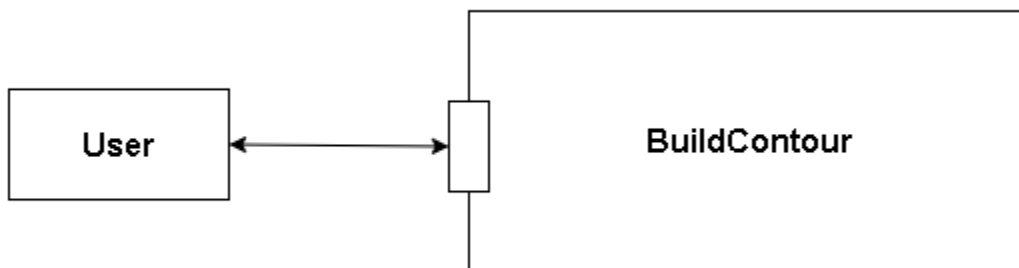


Figure 4: Architectural Context Diagram

6.2 Defining Archetypes

An archetype is a basic structural pattern that describes how a system's main parts work together to accomplish its functionality. By capturing recurrent design concepts that hold true even when implementation details change, archetypes aid in the simplification of architectural reasoning. Finding the right archetypes makes it simpler to comprehend, evaluate, and change the system's overall behavior and structure.

Building roof point cloud data passes through a number of clearly defined processing stages in the BandContour system, which is built as a data-driven processing pipeline. After completing a particular transformation on the data, each stage transfers the outcome to the subsequent stage. Modularity, data flow clarity, and ease of maintenance are guaranteed by this method.

Suitable Archetype: Pipe and Filter Architecture

The BandContour system follows the **Pipe and Filter architectural pattern**. In this architecture, the system is composed of a sequence of independent processing components called **filters**, which are connected by **pipes**. Pipes act as data channels through which processed data flows from one filter to another.

This archetype is particularly suitable for BandContour because roof contour extraction involves multiple sequential operations such as preprocessing, contour extraction, refinement, visualization, and area computation. Each operation can be cleanly isolated into a separate processing unit without tightly coupling it to other stages.

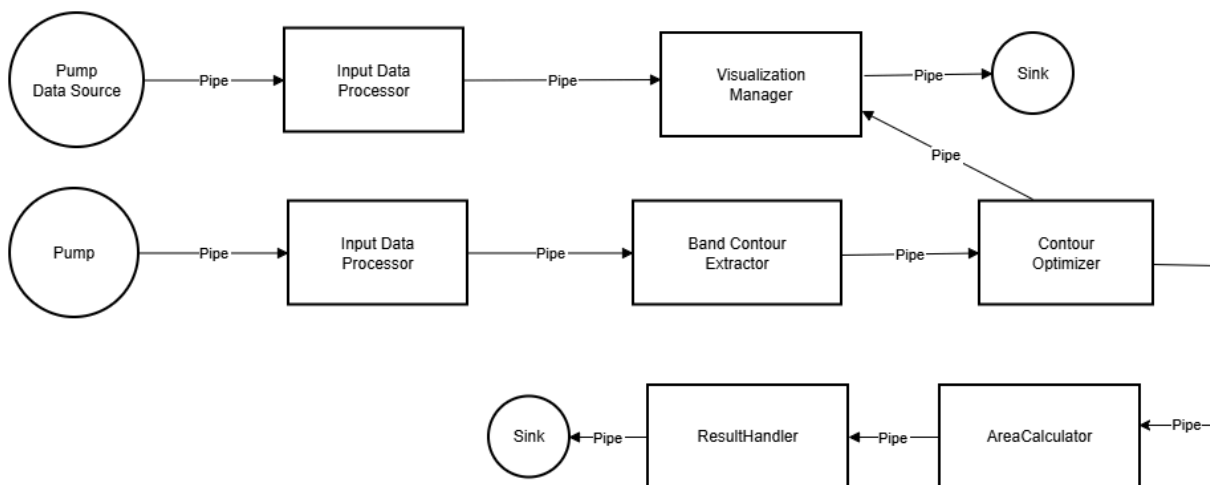


Figure 5: Archetype

6.3 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown below:

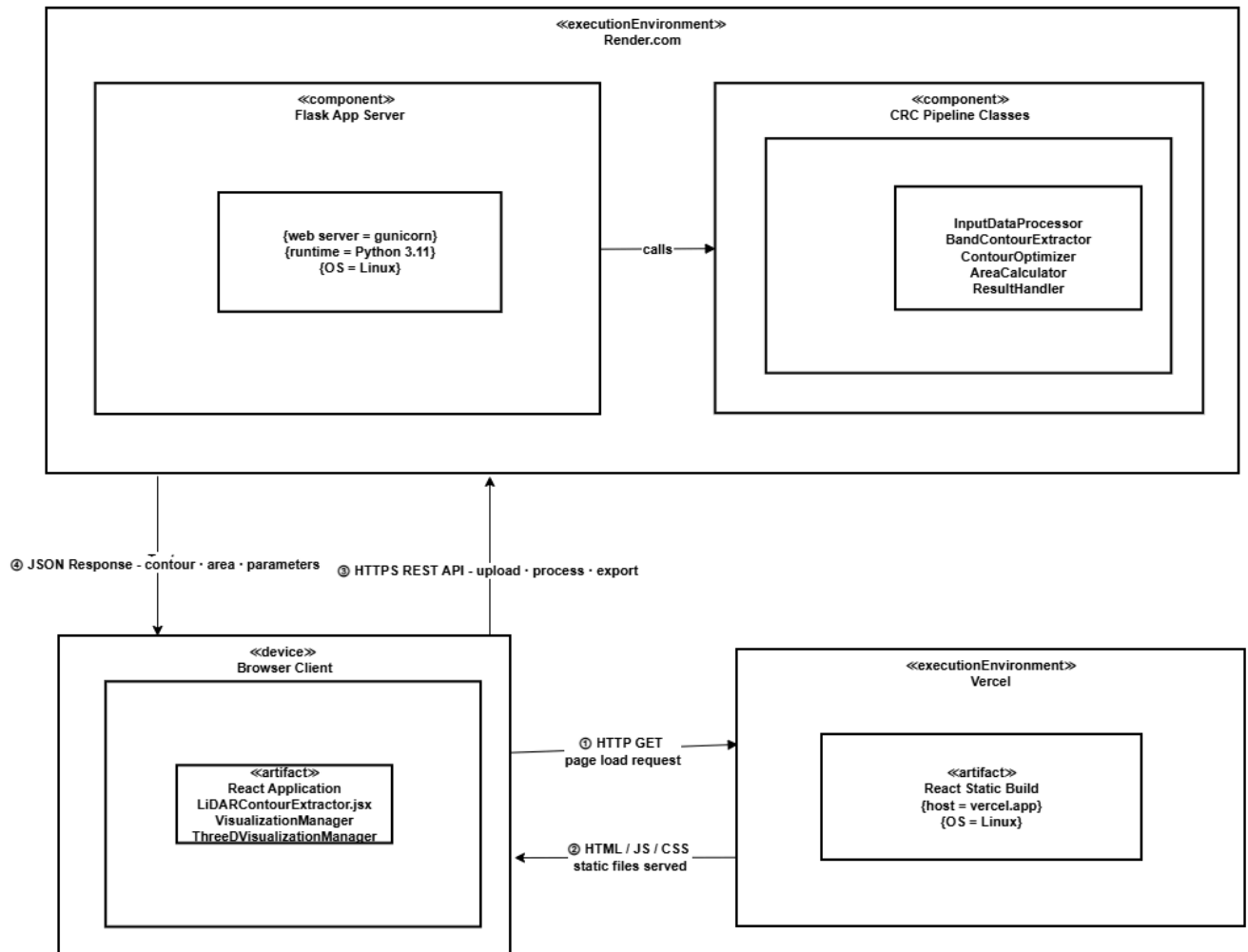
Software Component	Mapped Requirements
Point Cloud Processor	NR1: Data Input and Validation NR2: Roof Contour Extraction (Preprocessing) ER4: Performance Feedback
Contour Extractor	NR2: Roof Contour Extraction ER4: Performance Feedback
Visualizer	NR3: Visualization of Results ER2: Enhanced Visualization Controls ExcR1: Advanced Visualization and Comparison
Area Calculator	NR4: Area Calculation
Result Handler	NR4: Area Calculation NR5: Quantitative Evaluation and Comparison ER3: Result Export and Reporting
Export Manager	ER3: Result Export and Reporting ExcR3: Extended Output and Integration

6.4 Deployment diagram

A simple and unambiguous deployment architecture is necessary for the LiDAR Building Contour Extractor to be deployed successfully. This guarantees ease of implementation, scalability, and maintainability. A simplified deployment diagram divided into Frontend and Backend sections can be found below.

There are no external service dependencies in the system's two-tier client-server architecture. The frontend is delivered to the user's browser via HTTPS and is hosted on Vercel as a static React build. After loading, the browser uses a collection of REST API endpoints to connect directly to the Backend, which is a Python Flask application hosted on Render.com. There is no

direct connection between Vercel and Render.com; instead, the browser client initiates all communication. The LiDAR Building Contour Extractor's deployment diagram is given below:



7. User Interface Design

User interface design creates an effective communication medium between a human and a computer. Following a set of interface design principles, the design identifies interface objects and actions and then creates a screen layout that forms the basis for a user interface prototype.

Interface Analysis

In the case of user interface design, understanding the problem means understanding:

- The people (end users) who will interact with the system through the interface

- The tasks that end users must perform to do their work
- The content that is presented as part of the interface and
- The environment in which these tasks will be conducted

In the sections that follow, we examine each of these elements of interface analysis with the intent of establishing a solid foundation for the design tasks that follow.

7.1 User Analysis

The primary users of the LiDAR Building Contour Extractor are researchers, engineers, and students working in the fields of remote sensing, photogrammetry, and geospatial analysis. Specifically, the system is designed for:

- **Geospatial researchers** who need to extract accurate building footprints from airborne LiDAR data for urban modelling studies.
- **Civil and urban planners** who require roof area measurements and building boundary data for land use analysis.
- **Students and academics** studying the Wang et al. (2024) multidirectional banding algorithm and who wish to observe each processing step visually.
- **Software developers** integrating the extraction API into larger GIS pipelines who need a quick visual validation tool.

Users are expected to have basic familiarity with web browsers and point cloud data formats (.pts, .xyz, .las, .laz). No programming knowledge is required to operate the interface. The system runs entirely in a browser and requires no local installation beyond accessing the deployed Vercel URL.

7.2 Task Analysis

The following user tasks represent the complete set of interactions a user may perform within the LiDAR Building Contour Extractor. Objects are shown in boldface and actions in italics throughout this section.

7.2.1 User Task: Upload Point Cloud File

- Click on the **Upload Point Cloud panel**
- Click the **file input area** to open the system file browser
- Select a valid **point cloud file** in .pts, .xyz, .txt, .las, or .laz format
- Observe the **file name** and **point count** displayed in the upload panel

7.2.2 User Task: View LAS/LAZ File Metadata

- Upload a **LAS or LAZ binary file**
- Observe the **LAS Header Info box** appearing below the upload area
- Read the **version**, **point format**, **point count**, **scale**, and **offset values**

7.2.3 User Task: Extract Building Contours

- Click the **Extract Contours button** in the Process panel
- Observe the **loading spinner** while the backend processes the data

- View the **parameters panel** showing computed values for **d**, **W**, and **T**

7.2.4 User Task: View Algorithm Parameters

- Observe the **parameters box** that appears after extraction completes
- Read the **number of buildings found**
- Read the **average spacing d**, **band width W**, and **threshold T**

7.2.5 User Task: Select a Building

- View the **Select Building dropdown** that appears when multiple buildings are detected
- Click the **dropdown** to expand the list of detected buildings
- Select the desired **building entry** to update the active contour

7.2.6 User Task: View Building Statistics

- Observe the **Building Stats panel** in the left column
- Read the **input point count**, **contour point count**, and **roof area in m²**

7.2.7 User Task: Switch to 2D Visualization View

- Click the **"2D View" tab** in the right panel
- Click the **"Points Only" button** to view only the raw point cloud
- Click the **"With Contour" button** to overlay the extracted contour on the point cloud
- Observe the **stats box** in the top-left corner showing point count, contour count, and area

7.2.8 User Task: Switch to 3D Visualization View

- Click the **"3D View (Three.js)" tab** in the right panel
- Drag to rotate the **3D scene**
- Scroll to zoom in and out of the **3D point cloud**
- Right-drag to pan the **3D scene**
- Observe the **elevation colour gradient legend** indicating low to high Z values

7.2.9 User Task: Switch Back to 2D View

- Click the **"2D View" tab** to return from the 3D view
- Observe that the **2D canvas** retains all previously rendered points and contour

7.2.10 User Task: Export Contour as Text File

- Click the **"Export as .txt" button** in the Export Results panel
- Observe the browser **file download dialog** opening automatically
- Save the **contour.txt file** containing X Y coordinate pairs

7.2.11 User Task: Export Contour as JSON

- Click the **"Export as .json" button**
- Save the downloaded **contour.json file** containing the contour coordinate array

7.2.12 User Task: Export Contour as GeoJSON

- Click the **"Export as .geojson" button**

- Save the downloaded **contour.geojson file** as a GeoJSON Polygon Feature for use in GIS software

7.2.13 User Task: Export Visualization as PNG Image

- Click the "**Export PNG (VisualizationManager)**" button in the Export Results panel
- Save the downloaded **contour_visualization.png file** capturing the full 2D canvas

7.2.14 User Task: View Error Messages

- Observe the **error toast notification** that appears at the bottom-right of the screen for invalid inputs or server errors
- Click the **X (dismiss) button** on the error toast to clear the message

7.2.15 User Task: Re-run Extraction on Same Data

- Click the **Extract Contours button** again without re-uploading the file
- Observe that the previously loaded **point data** is reused from browser state
- Observe updated **contour results** and **statistics** in the panels

Objects (boldface) and actions (italics) are extracted from this list of tasks. This representation of objects (boldface) and actions (italics) are also maintained in the next section.

Objects	Actions
Upload Point Cloud panel	click, select
Point cloud file	upload, select
File name	observe, view
Point count	observe, read
LAS Header Info box	observe, read
Version	read
Point format	read
Scale	read
Offset values	read
Extract Contours button	click
Loading spinner	observe
Parameters box	view, read
Average spacing d	read
Band width W	read
Threshold T	read
Number of buildings found	read

Select Building dropdown	click, select
Building entry	select
Building Stats panel	observe, read
Input point count	read
Contour point count	read
Roof area in m ²	read
" 2D View" tab	click
"Points Only" button	click
"With Contour" button	click
Stats box	observe, read
2D canvas	observe
" 3D View (Three.js)" tab	click
3D scene	drag, scroll, pan, rotate
3D point cloud	zoom, rotate, pan
Elevation colour gradient legend	observe
"Export as .txt" button	click
File download dialog	observe
contour.txt file	save
"Export as .json" button	click
contour.json file	save
"Export as .geojson" button	click
contour.geojson file	save
"Export PNG" button	click
contour_visualization.png file	save
Error toast notification	observe
X (dismiss) button	click
Point data	reuse
Contour results	observe
Statistics	observe

7.3 User Manual

This manual provides step-by-step instructions for using the LiDAR Building Contour Extractor web application. The system allows users to upload building roof LiDAR point cloud files, automatically extract building boundary contours, visualize results in both 2D and 3D, and export the outputs in multiple formats.

7.3.1 System Requirements

No installation is required. The application runs entirely in a web browser. The following minimum requirements apply:

Web Browser	Google Chrome 100+, Mozilla Firefox 100+, Microsoft Edge 100+, or Safari 15+
Internet	Active internet connection to reach the deployed backend server
Screen	Minimum 1280 × 768 resolution recommended for full panel visibility
File Formats	.pts, .xyz, .txt (text), .las, .laz (binary LiDAR)
Max File Size	200 MB per upload
Access URL	https://lidar-frontend-zeta.vercel.app/

7.3.2 Application Layout

The application is divided into two main areas:

- **Left Panel** - Contains all control panels: Upload, Process, Building Selector, Building Stats, and Export.
- **Right Panel** - Contains the visualization area with 2D and 3D view tabs, and the Algorithm Pipeline reference card.

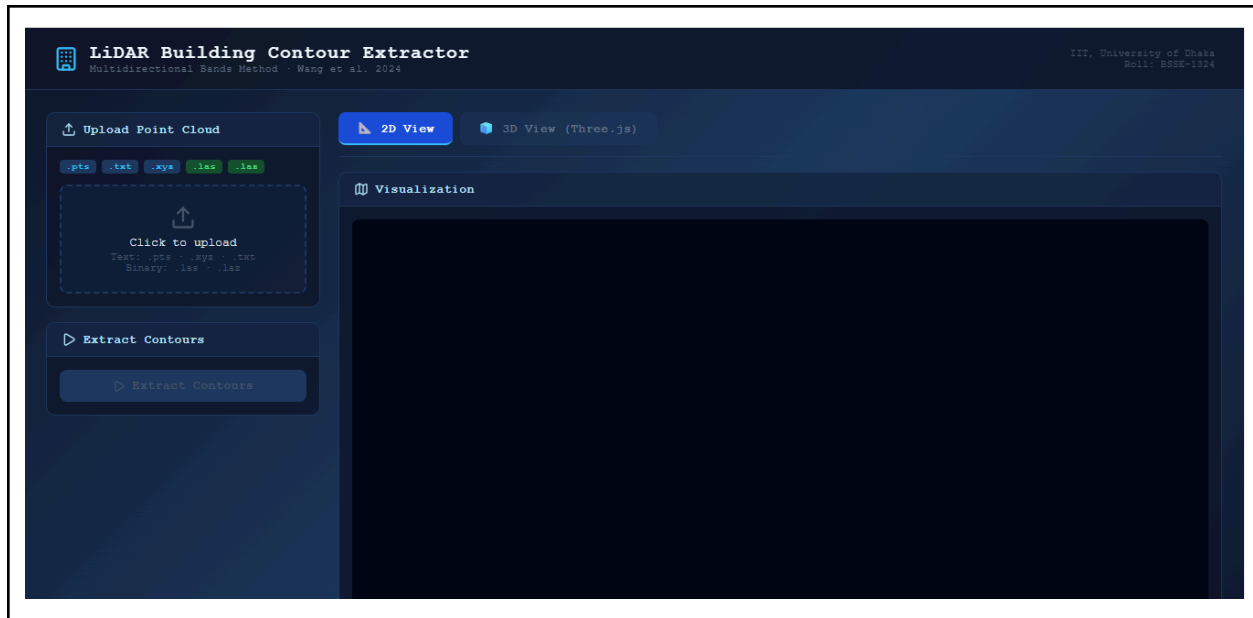


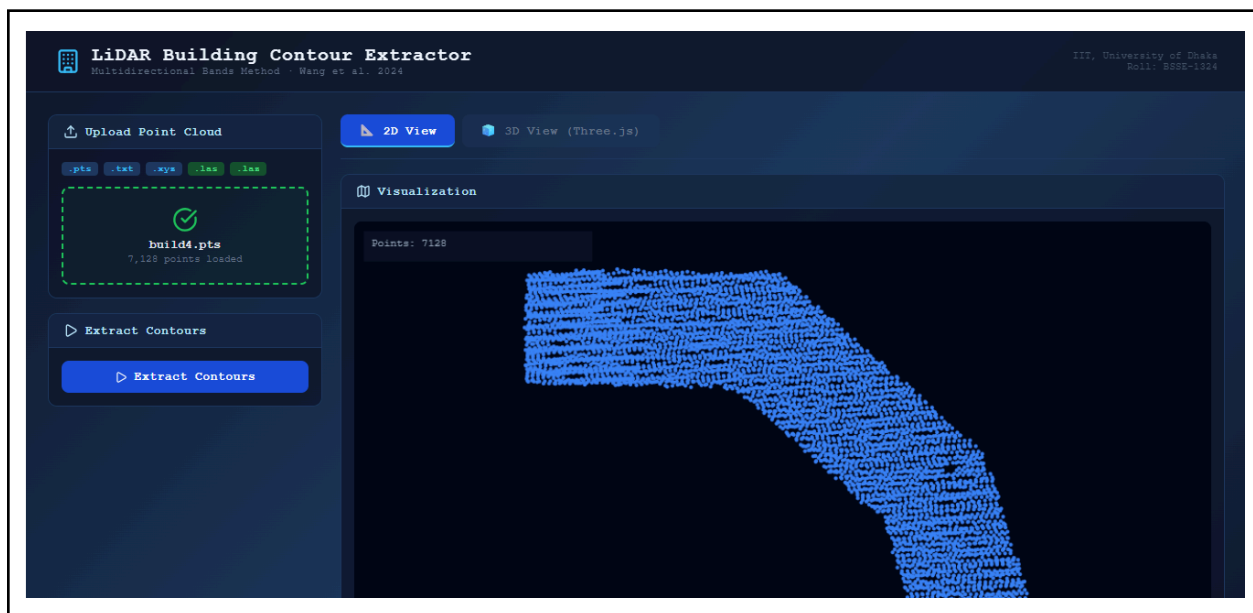
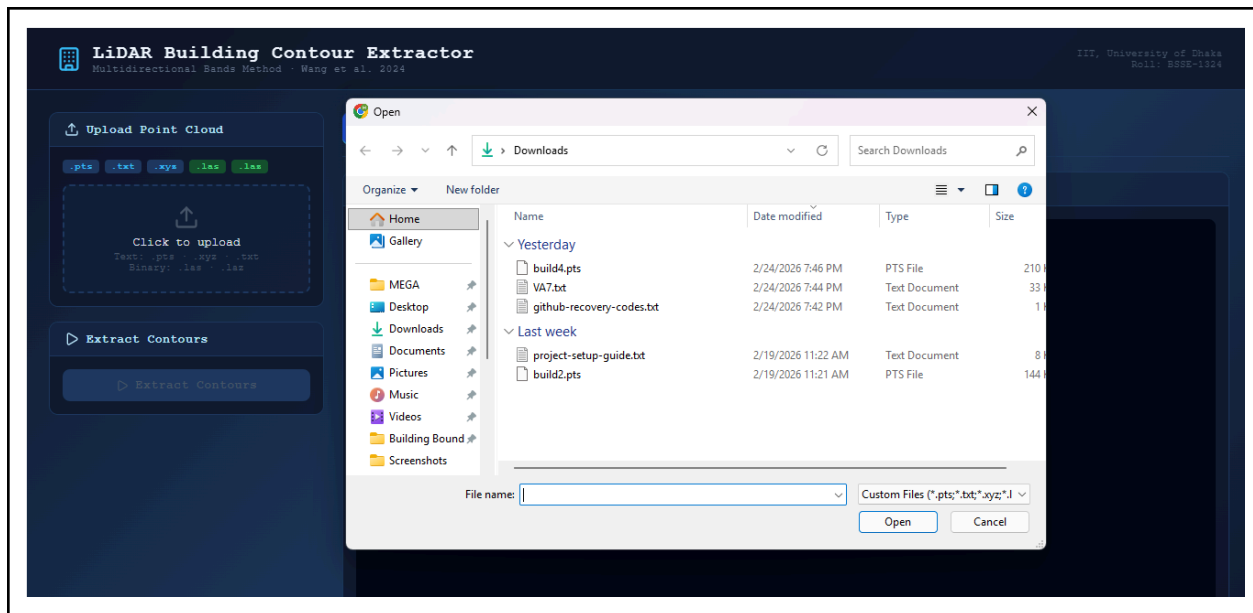
Figure: Overall application layout showing left control panel and right visualization panel

7.3.3 Uploading a Point Cloud File

The first step is to load a LiDAR point cloud file into the system. The application supports both plain text formats and binary LiDAR formats.

Steps to Upload

- Open the application in your browser.
- Locate the **Upload Point Cloud** panel in the top-left area of the screen.
- Click anywhere inside the dashed upload area. A system file browser dialog will open.
- Navigate to your point cloud file and select it.
- The upload area will display a green checkmark (✓), the file name, and the total number of points



7.3.4 Extracting Building Contours

After uploading a file, the system is ready to run the contour extraction pipeline based on the Wang et al. (2024) multidirectional banding method.

Steps to Extract

1. Locate the **Extract Contours** panel below the Upload panel.

2. Click the **Extract Contours** button. The button will show a spinning loader while the backend processes the data.
3. Wait for processing to complete. This typically takes 2 to 10 seconds depending on the number of points.
4. Once complete, the **parameters box** appears below the button showing:
 - **Buildings Found** - number of separate buildings detected
 - **Avg spacing d** - average nearest-neighbour point spacing in metres
 - **Band width W** = $8 \times d$ - width of each banding strip
 - **Threshold T** = $10 \times d$ - maximum allowed edge length before densification



Figure: Extract Contours panel after processing - showing parameters box with d, W, T values and buildings found

7.3.5 Using the 2D Visualization

The 2D View renders the point cloud and the extracted contour on an interactive canvas. It is the default view after extraction.

Switching to 2D View

1. Click the **2D View** tab at the top of the right panel.
2. Click the **Points Only** button to display only the raw point cloud in blue.
3. Click the **With Contour** button to overlay the extracted contour (shown as a red line with red vertex dots) on top of the point cloud.

Reading the Stats Box

A semi-transparent stats box is drawn in the top-left corner of the canvas. It displays:

- **Points** - total number of points in the uploaded file
- **Contour** - number of contour points for the selected building
- **Area** - roof area in m² for the selected building

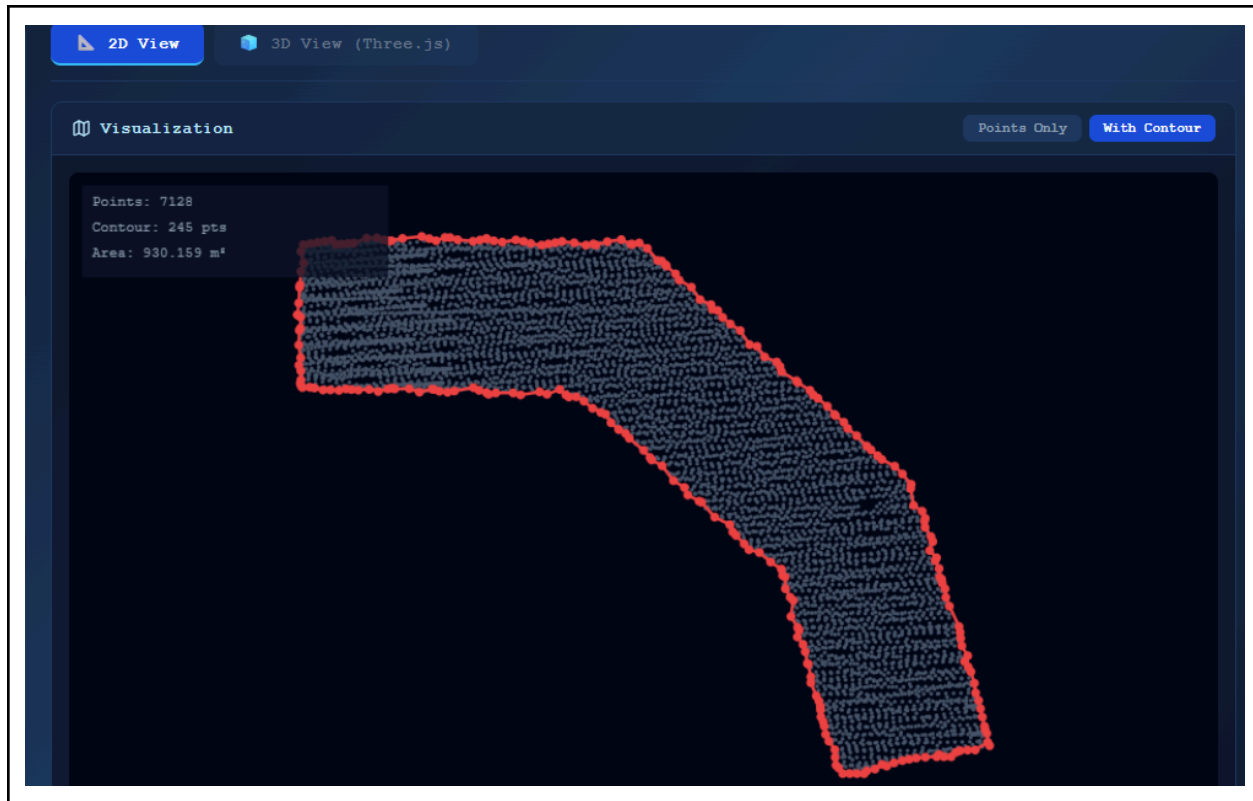


Figure: 2D View - Points Only mode showing raw point cloud in blue

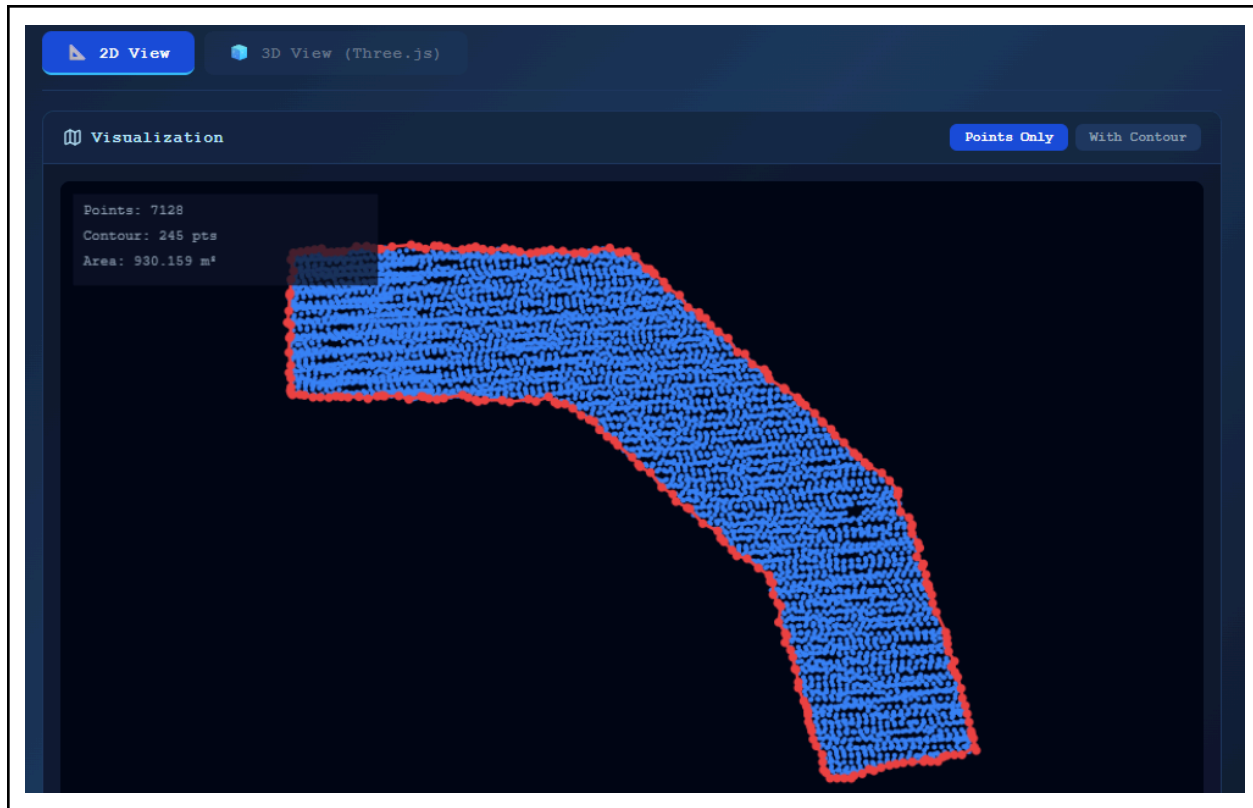


Figure: 2D View - With Contour mode showing red contour overlaid on grey points

7.3.6 Using the 3D Visualization

The 3D View renders the full point cloud in a Three.js scene with elevation-based colouring and interactive camera controls.

Switching to 3D View

1. Click the **3D View (Three.js)** tab at the top of the right panel.
2. The Three.js scene initializes and renders the point cloud automatically.

Camera Controls

Rotate	Left-click and drag anywhere in the 3D canvas
Zoom	Scroll the mouse wheel forward (zoom in) or backward (zoom out)
Pan	Right-click and drag to move the view laterally

Elevation Colour Gradient

Points are coloured according to their Z (elevation) value using a 5-stop gradient:

- Blue - lowest elevation

- Cyan - lower-mid elevation
- Green - mid elevation
- Yellow - upper-mid elevation
- Red - highest elevation

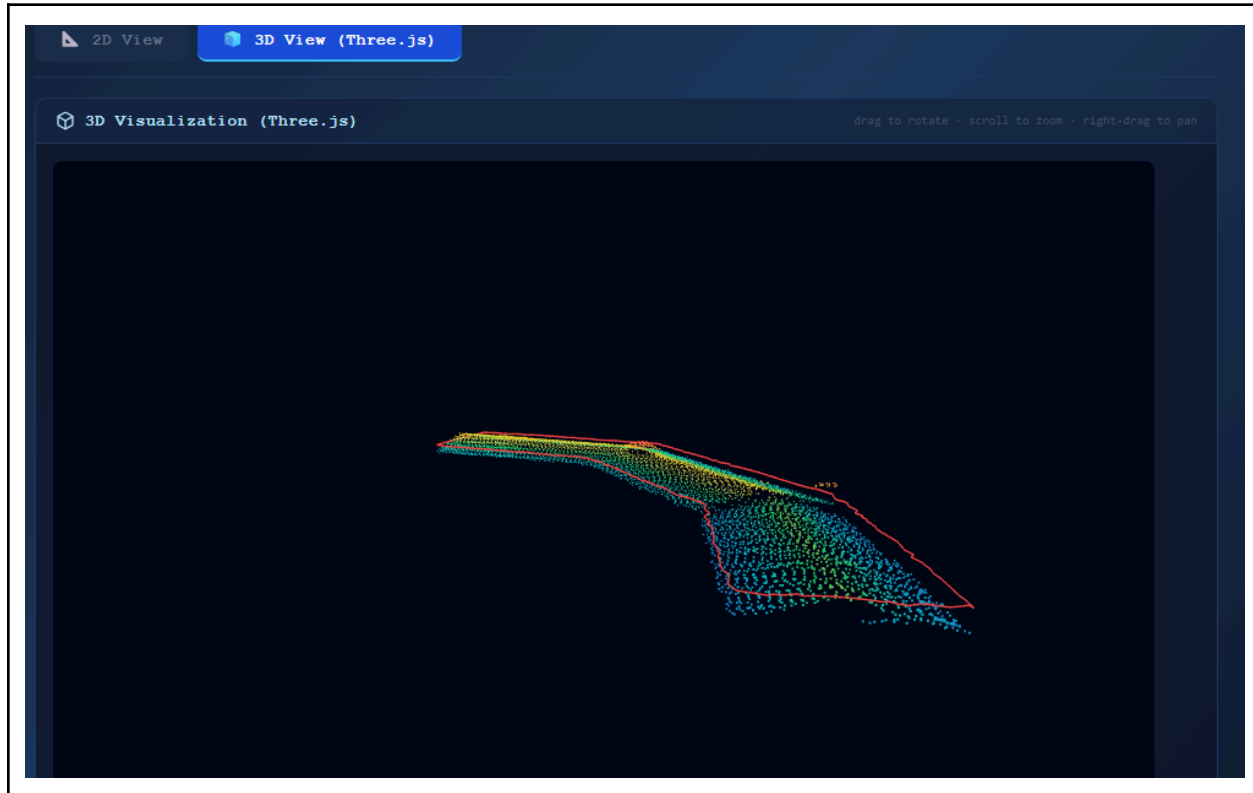


Figure: 3D View - point cloud with elevation colour gradient (blue = low, green = mid, red = high)

Switching Back to 2D View

1. Click the **2D View** tab.
2. The 2D canvas re-renders automatically. All points and the contour remain intact - no re-upload or re-processing is needed.

7.3.7 Exporting Results

The Export Results panel provides four download options. It appears in the left panel after extraction is complete.

Export as TXT

- Click the **Export as .txt** button.
- The browser will download a file named **contour.txt**.
- The file contains a comment header followed by one X Y coordinate pair per line for each contour point.

Export as JSON

- Click the **Export as .json** button.
- The browser will download a file named **contour.json** containing a JSON object with a contour coordinate array.

Export as GeoJSON

- Click the **Export as .geojson** button.
- The browser will download a file named **contour.geojson**, which is a valid GeoJSON Polygon Feature that can be opened directly in QGIS, ArcGIS, or other GIS software.

Export Visualization as PNG Image

- Click the **Export PNG (VisualizationManager)** button.
- The browser will download a file named **contour_visualization.png** that captures the entire 2D canvas including points, contour, and the stats box.



Figure: Export Results panel showing all four export buttons

7.3.8 Understanding Error Messages

If an error occurs during upload or processing, a red error toast notification appears in the bottom-right corner of the screen.

Dismissing an Error

- Click the **X button** on the top-right corner of the red error toast to dismiss it.
- The application remains fully functional after dismissing the error. No page refresh is required.

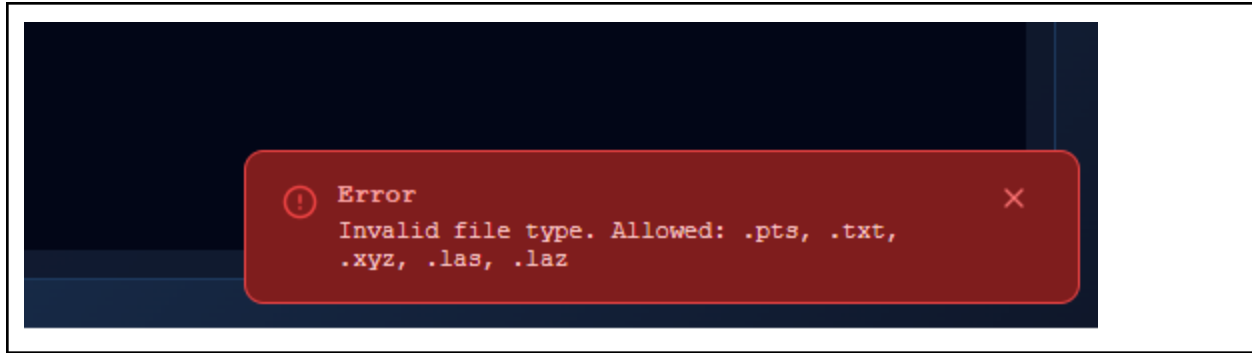


Figure: Error toast notification at bottom-right corner with dismiss X button

7.3.9 Re-running Extraction Without Re-uploading

The uploaded point cloud data is held in the browser's memory for the duration of the session. Users do not need to re-upload the file to run the extraction pipeline again.

- After a file has been uploaded, click the **Extract Contours** button at any time to re-run the pipeline on the same data.
- The results, stats, and visualization will update automatically.

8. Testing Components

This chapter presents a high-level overview of the testing objectives and outlines the major features that will be tested to ensure the correctness, reliability, and usability of the proposed **BandContour web-based roof contour extraction system**.

8.1 High-Level Description of Testing Goals

The primary testing goals of the BandContour system are as follows:

- To verify that the system satisfies all specified functional requirements
- To ensure that roof contour extraction produces accurate and consistent results
- To reduce false boundary detections and missing contour segments
- To validate correctness of computed geometric metrics such as area
- To ensure smooth and stable operation of the web application under typical usage

8.2 Testing Approach and Strategy

8.2.1 Testing Method

Black-box functional testing is the primary approach, where each test case inputs data to the system and verifies the output against expected outcomes without inspecting internal code execution. Component-level testing targets individual CRC classes.

8.2.2 Test Environment

Frontend	React 18 (CRA), Chrome 120+, localhost:3000
Backend	Python 3.11, Flask 3.x, localhost:5000
Libraries	numpy, scipy, laspy[lazrs], Three.js r128
Test Data	Toronto building LiDAR dataset (build2.pts) + synthetic rectangular roofs
Deployment	Render.com (backend) + Vercel (frontend)

8.2.3 Pass / Fail Criteria

- **PASS:** System produces the expected output for all stated expected outcomes with no application crash.
- **FAIL:** Any expected outcome is not met, or the application crashes, hangs, or throws an unhandled exception.
- **Partial Pass:** Some outcomes met but not all -treated as FAIL until all are resolved.

8.3 Detailed Test Cases

The following test cases describe the major testing scenarios for the system.

TC001 Building Point Cloud Input Validation	
Test Scenario	Verify that the system correctly accepts valid building point cloud files and rejects unsupported or corrupted inputs.
Input / Precondition	Valid files: sample.pts, sample.txt, sample.las, sample.laz. Invalid files: image.png, document.pdf, empty.txt, corrupted.pts.
Steps	1. Upload a valid .pts file through the Upload Point Cloud panel. 2. Verify point count is displayed and the Extract Contours button becomes active. 3. Repeat with .xyz, .txt, .las, and .laz formats. 4. Attempt to upload an unsupported format (e.g., .png). 5. Verify the error toast message appears with a descriptive error. 6. Attempt to upload an empty or corrupted file and observe system response.
Expected Outcomes	• Valid point cloud files (.pts, .xyz, .txt, .las, .laz) are successfully uploaded and parsed. • An appropriate error toast message is shown for unsupported or empty files.

	The web application remains stable without crashing.
Result	Passed - All valid formats accepted. Error messages correctly displayed for invalid inputs.

TC002 2D Contour Visualization	
Test Scenario	Ensure that the extracted contour is correctly visualized over the original point cloud in 2D view.
Input / Precondition	Any uploaded and processed point cloud with at least one detected building.
Steps	1. Upload a point cloud and process it. 2. Switch between "Points Only" and "With Contour" view modes. 3. Verify original points and contour are displayed together in overlay mode. 4. Confirm the stats box shows: total points, contour point count, and area. 5. Switch to 3D View and back to 2D View. 6. Verify all points and contour remain visible after switching back (no blank canvas).
Expected Outcomes	- Original points rendered in blue (overlay mode) or darker colour (points-only mode). - Extracted contour rendered as a red line with red vertex dots. - The stats box in the top-left corner shows Points, Contour pts, and Area correctly. - After switching from 3D to 2D and back, the canvas re-renders correctly. - No blank canvas or missing points after tab switching.
Result	Passed - 2D visualization renders correctly including after 3D/2D tab switching.

TC003 3D Point Cloud Visualization (Three.js)	
Test Scenario	Verify that the 3D visualization correctly renders point cloud and contour using Three.js with elevation-based colouring.
Input / Precondition	Any uploaded point cloud with Z coordinate data.
Steps	1. Upload a 3D point cloud and process it. 2. Switch to the "3D View (Three.js)" tab. 3. Verify the point cloud appears with elevation-based colour gradients (blue to red). 4. Drag to rotate the scene and verify OrbitControls work. 5. Scroll to zoom in/out.

	- Right-drag to pan the scene. - If a contour was extracted, verify it appears as a red line loop. - Switch back to 2D View and verify 2D canvas is still intact.
Expected Outcomes	- Point cloud rendered with elevation colour gradient (blue=low, green=mid, red=high). - OrbitControls allow rotation, zoom, and panning without errors. - Extracted contour displayed as a red 3D line loop on the point cloud. - WebGL context remains stable throughout interaction. - Switching back to 2D View does not break either panel.
Result	Passed - 3D visualization renders correctly. OrbitControls functional. Contours visible.

TC004 Roof Area Computation	
Test Scenario	Verify that the system accurately computes the area enclosed by the extracted roof contour using the Shoelace formula.
Input / Precondition	A synthetic point cloud representing a rectangular roof of known dimensions (e.g., 20m × 15m = 300 m²).
Steps	- Upload a synthetic rectangular roof point cloud with known dimensions. - Run the extraction pipeline. - Check the Building Stats panel for the computed roof area. - Verify the area value matches the expected geometric area. - Confirm the area is also shown in the 2D View stats box (top-left corner). - Repeat with a triangular roof and verify the area.
Expected Outcomes	- The computed area matches the expected geometric values within acceptable tolerance. - Area result is clearly displayed in the Building Stats panel as area_m2. - The 2D visualization stats box shows: Points, Contour pts, and Area in m². - The Shoelace formula produces consistent results across multiple runs.
Result	Passed - Area computed correctly. Displayed in both stats panel and 2D canvas.

TC005 Result Export Functionality
--

Test Scenario	Test the system's ability to export contour and analysis results in multiple formats.
Input / Precondition	A processed building contour ready for export.
Steps	1. Upload and process a point cloud to produce a contour. 2. Click "Export as .txt" and verify the downloaded file contains X Y coordinate pairs. 3. Click "Export as .json" and verify the downloaded JSON contains a contour array. 4. Click "Export as .geojson" and verify the file contains a valid GeoJSON Polygon. 5. Click "Export PNG (Visualization)" and verify the 2D canvas is saved as an image. 6. Open each exported file and confirm it matches the on-screen result.
Expected Outcomes	• TXT export contains one X Y coordinate pair per line with a header comment. • JSON export contains a valid JSON object with a contour array. • GeoJSON export is a valid GeoJSON Feature with a Polygon geometry. • PNG export captures the full 2D canvas including points, contour, and stats box. • All exported files match the displayed on-screen results.
Result	Passed - All four export formats produce correct, matching output files.

TC006 Error Handling and Application Stability	
Test Scenario	Verify system behaviour under incorrect inputs, network failures, and processing errors.
Input / Precondition	Corrupted files, empty files, wrong extensions, disconnected backend.
Steps	1. Upload a file with an unsupported extension (e.g., .csv). 2. Upload an empty .pts file with no data points. 3. Upload a valid file but stop the Flask backend and attempt to process. 4. Attempt to click "Extract Contours" without uploading a file first. 5. Submit a request with a file containing only header lines and no numeric data. 6. After each error, verify the application remains usable without refreshing.
Expected Outcomes	• Clear error toast messages are shown for each invalid scenario. • The error toast can be dismissed with the X button.

	• The Extract Contours button is disabled when no file is loaded. • Network/backend errors show a descriptive "Cannot reach backend" message. • The application remains stable without crashing or requiring a page refresh.
Result	Passed - All error conditions handled gracefully. Application remains stable.

TC007 Result Regeneration Without Re-Upload	
Test Scenario	Verify that users can change building selection and regenerate visualization without re-uploading data.
Input / Precondition	Previously uploaded and processed point cloud data held in React state.
Steps	1. Upload a point cloud and run the extraction pipeline. 2. Switch between buildings in the selector dropdown. 3. Toggle between "Points Only" and "With Contour" view modes. 4. Switch between 2D and 3D views multiple times. 5. Verify all interactions use the already-loaded points without making new upload requests.
Expected Outcomes	• Previously uploaded point data is reused from React state without re-uploading. • Changing building selection immediately updates the canvas without server calls. • View mode toggling is instant as it only re-renders the existing data. • No additional network requests are made for visualization changes.
Result	Passed - All visualization interactions use cached state. No unnecessary server calls.

TC008 Average Point Spacing Calculation	
Test Scenario	Verify that the system correctly computes the average point spacing (d) using the k=80 anchor point method as specified in Wang et al. (2024).
Input / Precondition	A point cloud file with known point density (e.g., build2.pts with approximately 0.05m average spacing).
Steps	1. Upload a known point cloud file. 2. Click the Extract Contours button to trigger the pipeline. 3. Observe the parameters panel showing avg spacing d, band width W, and threshold T. 4. Verify $W = 8 \times d$ and $T = 10 \times d$.

	5. Repeat with a sparser point cloud and verify d increases proportionally.
Expected Outcomes	• The average spacing value (d) is calculated and displayed in the parameters panel. • Band width W is correctly derived as $8 \times d$. • Densification threshold T is correctly derived as $10 \times d$. • Values change proportionally when input point density changes.
Result	Passed - d, W, and T computed correctly. $W = 8d$ and $T = 10d$ relationship verified.

TC009 Multidirectional Band Contour Extraction

Test Scenario	Verify that the system extracts contour points using 6 banding directions (0°, 30°, 60°, 90°, 120°, 150°) as specified in the paper.
Input / Precondition	A building roof point cloud with a complex non-rectangular shape.
Steps	1. Upload a complex-shaped building point cloud. 2. Run the extraction pipeline. 3. Switch to 2D View and enable "With Contour" mode. 4. Observe that boundary points are detected on all sides of the building. 5. Verify that concave corners of the building are captured in the contour.
Expected Outcomes	• Boundary points are detected from all 6 specified directions. • Contour points represent the complete outer roof boundary including concave areas. • No significant gaps exist in the extracted contour. • The number of contour points shown in the stats box is consistent with the building complexity.
Result	Passed - All 6 directions produced contour points. Boundary coverage verified visually.

TC010 Contour Point Sorting with $\beta=240^\circ$ Constraint

Test Scenario	Validate that unordered contour points are sorted into a continuous sequence using the forward-direction constraint ($\beta=240^\circ$).
Input / Precondition	Unsorted contour points from the BandContourExtractor output.
Steps	1. Upload a building point cloud and run the extraction pipeline. 2. View the contour in 2D View with "With Contour" mode enabled. 3. Verify the contour forms a single continuous closed loop.

	- Inspect for any backward jumps or crossing lines in the contour. - Test with a building that has a complex outline to stress-test the constraint.
Expected Outcomes	- Contour points are ordered correctly without backward jumps or crossings. - A continuous, closed contour structure is formed. - The $\beta=240^\circ$ constraint prevents the algorithm from connecting to points behind the current direction. - The contour visually follows the building boundary without erratic jumps.
Result	Passed - Sorted contour forms a clean closed loop with no visible crossing edges.

TC011 Long Edge Densification (T = 10d)	
Test Scenario	Verify that edges longer than the threshold $T=10d$ are identified and densified by inserting real point cloud points.
Input / Precondition	A building point cloud where some sections have sparse contour coverage.
Steps	- Upload a point cloud with areas of low boundary point density. - Run the extraction pipeline and note the initial contour point count. - Verify the final contour point count is higher than the sorted contour count. - Inspect the contour for uniform edge distribution in 2D View. - Verify that inserted points come from the original point cloud (not interpolated).
Expected Outcomes	- Long edges ($> T=10d$) are subdivided by inserting actual point cloud points. - The final contour has more points than the unsorted contour. - Edge lengths are more uniform across the entire contour boundary. - No artificially interpolated points are inserted -only real cloud points are used.
Result	Passed - Densification increased contour point count. Edge distribution is more uniform.

TC012 Noise Point Removal (Elevation Threshold)	
Test Scenario	Verify that contour points with anomalous elevation (parapet walls, antennas) are removed based on 5-neighbour elevation comparison.

Input / Precondition	A 3D point cloud where the roof boundary includes elevated parapet wall sections.
Steps	1. Upload a point cloud containing roof features at different elevations. 2. Run the full pipeline including noise removal. 3. Compare contour point count before and after noise removal. 4. Verify that the remaining contour points represent the main roof elevation. 5. Check that no points with large elevation deviations remain in the final contour.
Expected Outcomes	• Points whose elevation deviates more than 5×d from their 5 nearest neighbours are removed. • The final contour represents a consistent elevation profile. • Parapet wall and antenna points are excluded from the contour. • The contour point count is reduced compared to the densified contour.
Result	Passed - Anomalous elevation points removed. Final contour follows main roof surface.

TC013 LAS/LAZ Binary File Support	
Test Scenario	Verify that the system correctly reads LAS and LAZ binary point cloud files using laspy and applies the stored scale and offset.
Input / Precondition	A standard LAS 1.2 or LAS 1.4 file, and a LAZ compressed version of the same data.
Steps	1. Upload a .las file through the upload panel. 2. Verify the "LiDAR binary format" badge appears on the file entry. 3. Verify the LAS Header Info box shows version, point format, point count, scale, and offset. 4. Run the extraction pipeline and verify results match the expected building layout. 5. Upload the LAZ compressed version of the same dataset. 6. Verify identical point counts and bounds for both LAS and LAZ versions.
Expected Outcomes	• LAS files parsed correctly with scale and offset applied from the file header. • LAZ compressed files decompressed and parsed identically to LAS. • LAS Header Info panel displays version, point format, count, scale, and offset. • Point cloud coordinates are in real-world units (metres or feet as stored in file). • The extraction pipeline produces equivalent results for both LAS and LAZ inputs.
Result	Passed - LAS and LAZ files read correctly. Header metadata displayed. Pipeline unaffected.

9. Conclusion

The BandContour web-based building roof contour extraction system's full set of requirements, design factors, and testing plan are presented in this document. The project's motivation, requirements classification, usage scenarios, architectural design, and initial testing plan are all covered in the report.

The main goal of the suggested system is to use a multidirectional band-based methodology to directly extract precise roof contours from building point cloud data. The system prioritizes efficiency, robustness, and interpretability by eschewing rasterization and significant machine learning dependencies.

Overall, this report outlines the project's development roadmap, workflow, and validation approach. Although the main goals of the current design are met, future improvements could enhance automation, scalability, and integration with 3D city modeling platforms.

References

- [1] Wang, Jingxue et al. "Extraction of Building Roof Contours from Airborne LiDAR Point Clouds Based on Multidirectional Bands." *Remote. Sens.* 16 (2024): 190.
- [2] Zang, Dongdong et al. "Semantic Extraction of Roof Contour Lines From Airborne LiDAR Building Point Clouds Based on Multidirectional Equal-Width Banding." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 17 (2024): 16316-16328.
- [3] Liu K, Ma H, Li L, Huang S, Zhang L, Liang X, Cai Z. Building Outline Extraction via Topology-Aware Loop Parsing and Parallel Constraint from Airborne LiDAR. *Remote Sensing*. 2025; 17(20):3498. <https://doi.org/10.3390/rs17203498>